Drift Pattern Taxonomy

A behavioral drift pattern describes a trajectory condition that may indicate a departure from the intended workflow or unproductive progress. The taxonomy covers recurrence, exploration, workflow, tool execution, editing, validation, reasoning activity, explicit constraints, and completion. Related multi-agent findings are included where they transfer to the executor's own behavior. These patterns support diagnosis. A correctness judgment may require additional evidence.

Scope and interpretation

G denotes recorded Graphectory occurrences, attributes, and structural relations, including thought occurrences in the extended representation. L denotes Langutory phase labels and order, and G+L requires both. G* denotes additional evidence such as normalized arguments, typed results, state versions, contracts, provenance, or resource measurements. S denotes a learned behavioral drift that requires a judgment about the task or behavior and may need external evidence.

Source abbreviations identify the literature listed at the end. "Derived" marks an operational predicate or coding boundary motivated by that literature, rather than an identical detector reported by the source. Thresholds, windows, normalization, and comparability conditions must be declared. Missing evidence remains unknown; the pending action's outcome is unavailable before execution.

Assign each fact to its most specific supported description. Record outcomes, recurrence counts, and domain context as attributes rather than additional copies of the same incident. Event conditions and aggregate patterns may describe the same trajectory at different scales; distinct evidence dimensions should not be counted as independent failures without justification.

Family overview

Family	IDs	Count	Evidence
A Action and observation recurrence	A01–A03	3	Action identities, temporal motifs, equivalent feedback
B Repository exploration	B01–B05	5	Repository regions, hierarchy, view ranges
C Workflow structure and phase progression	C01–C04	4	Required milestones, phase order, phase motifs
D Tool invocation and execution	D01–D04	4	Interfaces, dispatch status, errors, rejection history
E Editing and code revision	E01–E07	7	Replacement arguments, edit outcomes, source versions
F Testing and validation	F01–F07	7	Test executions, selected tests, revision-matched outcomes

G Observable reasoning activity	G01–G02	2	Exposed thought occurrences and execution boundaries
H Context and explicit constraints	H01–H06	6	Allowed actions, state invariants, versioned references
I Completion and resource use	I01–I04	4	Final artifacts, current check results, completion and limits
S Learned Behavioral Drifts*	S01–S23	23	Judgment about the task or behavior, supported by external evidence where needed

The catalogue contains 65 behavioral drift patterns across ten families. G and L identify process facts represented in the trajectory. G* entries require the additional evidence stated in their predicates.

A Action and observation recurrence

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
A01	Action revisit	An action with the same recorded identity as an earlier executed action is proposed or executed again. Record the occurrence count within a declared window, including any threshold crossing. Matching identities may have different inputs or state.	G	GR §2.3; WI §2.1.2; Derived
A02	Action oscillation	An ordered sequence of two or more distinct action identities repeats. Record the motif and its repetitions. Count a single-identity loop under A01 and record unsuccessful outcomes as qualifiers.	G	WI §2.1.2; Derived
A03	Equivalent feedback across distinct actions	Distinct action identities produce equivalent, nonempty, non-error information payloads under a declared normalization. Repeated execution errors belong to D04.	G*	TA §III-B3; Derived

B Repository exploration

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source

B01	RepeatedView	A previously inspected structural region is inspected again. Record whether the same command or changed view ranges produced the return; unchanged content is an optional qualifier.	G	GR Table 3
B02	Scroll	Different view ranges on the same file overlap. An identical repeated range is recorded as RepeatedView rather than as another range-overlap incident.	G*	GR Table 3; Derived coding boundary
B03	ZoomOut	The temporal navigation sequence moves from a deeper repository region to an ancestor or shallower structural level. Record the source and destination regions.	G	GR Table 3
B04	OverlyDeepZoom	An extended sequence of views concentrates on one file or directory subtree. Subsequent patching may or may not occur; the prefix condition concerns concentration already observed.	G	GR Table 3, §3.4
B05	Dispersed exploration	Navigation repeatedly switches among separate repository subtrees within a declared interval. Measure cross-subtree dispersion; simple ancestor returns are ZoomOut rather than dispersion incidents.	G*	GR §3.3; IS C2.2; Derived

C Workflow structure and phase progression

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
C01	Premature validation	The agent enters validation before any patch activity under the declared localization–patch–validation order. Pre-patch bug reproduction is classified as a distinct activity.	L	PA §2; Derived

C02	Missing required reproduction	A declared workflow milestone is reached without its required reproduction activity. Apply only when reproduction is an explicit prerequisite for that milestone.	G+L	PA §§2,5.1; Derived
C03	Phase backtracking	A transition returns to an earlier phase under a declared phase order. Record the frequency of backward transitions and use C04 when those transitions form a recurring motif.	L	GR §3.3; PA §5.3; Derived
C04	Repeated phase motif	A subsequence of phase transitions recurs after adjacent identical phase labels are collapsed. This captures cyclic phase structure rather than same-phase dwell or exact action repetition.	L	GR §3.3; TA §III-B3; Derived

D Tool invocation and execution

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
D01	Invalid tool invocation	A selected call violates a known tool interface through an unavailable tool name, malformed executable syntax, missing argument, or invalid argument type. Ordinary reasoning text is not itself an invocation.	G*	AR §2.2; WI §2.1.3
D02	Execution or setup failure	A dispatched tool attempt reports a setup or external execution error, such as missing dependency, unavailable resource, authorization failure, service error, or timeout. Editor-target rejection and executed test failures are classified in E and F.	G*	TR §3.2; IS C1.1; AR §2.2; Derived grouping
D03	Repeated proposal after rejection	A proposal rejected before dispatch is proposed again without a relevant permission, policy, or prerequisite-state change. Compare the recorded rejection reason and applicable state; unexecuted attempts require a separate ledger.	G*	AR §2.2; AS §3; Derived

D04	Recurring execution error across distinct actions	Different action identities yield the same normalized setup or external-execution error. Record the common signature and relevant conditions. Matched test failures across source revisions are classified in F.	G*	WI §2.1.3; TR §3.2; Derived

E Editing and code revision

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
E01	NoEffectEdit	A literal replacement supplies identical old and new text under the editor's replacement rules. This establishes an intended no-op; execution may still reject the target.	G*	GR Table 3
E02	StrNotFound	An executed replacement is rejected because the specified old text is absent from the current target. The recorded rejection applies to that attempt.	G*	GR Table 3
E03	AmbiguousTarget	An executed replacement is rejected because its target matches multiple locations and the interface cannot select one unambiguously. Use the editor's actual matching rules and status.	G*	GR Table 3
E04	UnresolvedRetry	A consecutive segment of executed edit attempts on one file contains no successful modification. Attempts may have different identities; a successful modification ends the segment. Rejection causes qualify the segment.	G	GR Table 3
E05	EditReversion	A completed edit restores content changed by an earlier successful edit. Repeated version alternation is a reversion pattern; overlapping edits alone do not establish restoration.	G*	GR Table 3; Derived state comparison

E06	Dispersed patch targets	Recorded modifications span structurally distant files or regions within a declared revision interval. Measure spatial dispersion separately from the number or size of changes.	G*	IS C2.2; MS RQ1; Derived
E07	Patch-scope growth	Changed-file count or effective diff size increases across recorded revisions relative to a declared starting state. This measures growth in patch extent rather than spatial dispersion or restoration of old content.	G*	MS RQ1; WI §2.1.1; Derived

F Testing and validation

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
F01	Stale validation	Earlier validation exists, but later effective source changes lack subsequent applicable validation. Match checks to source revisions; phase order alone is a weaker proxy and does not establish a passing test.	G*	TA §III-B3; Derived
F02	Multiple revisions without test execution	More than one effective source revision is produced without an intervening test execution, possibly with exploration in between. Record revision boundaries and actual test dispatches rather than relying on patch-phase dwell.	G*	TA §III-B3; Derived revision predicate
F03	Repeated testing with unchanged inputs	A test invocation is rerun with matching recorded source, configuration, and relevant environment inputs. A failed rerun is an outcome qualifier. Input equality does not eliminate nondeterminism or flakiness.	G*	TA §III-B3; SP §3.2.1; Derived
F04	Persistent test failure across revisions	Matched tests on distinct source revisions produce the same normalized failure signature under comparable conditions. Exclude setup failures; persistence of one failure does not establish absence of all repair progress.	G*	IS C2.1; Derived

F05	Observed pass-to-fail transition	A matched test that passed on an earlier revision fails on a later revision under comparable recorded conditions. Exclude setup errors; the transition alone does not establish patch causation.	G*	IS §5.2 C; TA §III-C; Derived
F06	Test evidence modified after failure	A recorded failed check is followed by changes to its test code, assertions, fixtures, or configuration. Assessing whether the edit weakened the test oracle requires knowing the expected behavior.	G*	IS C2.3; Derived observable proxy
F07	Previously failing tests omitted	A later invocation selects a narrower comparable test set and omits previously failing tests. Code this as a selection change; removal through changes to test definitions belongs to F06.	G*	IS C2.3; MA FM-3.2; Derived

G Observable reasoning activity

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
G01	Thought oscillation	Recorded thought identities or exact textual motifs recur within a declared reasoning episode. Exact repetition is the period-one case; judging whether thoughts agree or conflict requires interpreting their content.	G	WI §2.1.2; Derived
G02	Prolonged reasoning without execution	An extended sequence of exposed reasoning messages contains no executable action or new external observation. When the same sequence meets G01, record recurrence as a qualifier of the reasoning episode.	G	WI §2.1.2; Derived

H Context and explicit constraints

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
H01	Explicit path-scope violation	An operation targets a path outside an explicitly permitted scope. Resolve the target sufficiently to decide membership; repository distance alone does not establish a scope restriction.	G*	WI §2.1.1; AS §3; Derived contract
H02	Explicit tool or operation violation	An otherwise well-formed call uses a tool or operation forbidden by an explicit machine-checkable restriction. Interface invalidity is D01; permission to invoke a tool does not establish task relevance.	G*	AS §3; AR §2.2; Derived contract
H03	Explicit state-invariant violation	A declared, machine-checkable state invariant is false, such as a requirement to preserve a named configuration value. This concerns required state properties rather than allowed paths, tool interfaces, phase order, or final artifacts.	G*	AS §3; AR §3.2; Derived
H04	Stale versioned reference	An operation uses a recorded file version, range anchor, or object handle that is demonstrably obsolete or invalid in the current state. Determining whether a belief is outdated requires comparing it with the current state.	G*	IS C3; TA §III-C; Derived
H05	Missing required reference provenance	A well-typed reference parameter has no permitted origin in a complete evidence ledger under an explicit provenance requirement. Exempt generative parameters; obsolete previously established references belong to H04.	G*	PG §§4.2–4.3; Derived restricted proxy
H06	Recorded context truncation or reset	Captured model-input records show that previously supplied history was removed or the visible context reset to an earlier state. Requires actual context logs; assessing whether a summary lost relevant information requires comparing it with the original context.	G*	MA FM-1.4, FM-2.1; TR §3.3; Derived observable proxy

I Completion and resource use

ID	Behavioral Drift Pattern	Observable predicate / intuition	Class	Source
I01	Required final artifact absent	At finalization, a declared required file, artifact, or effective source change is absent. Apply this pattern only when the task explicitly requires the missing result.	G*	SP §3.2.1; AS §3; Derived
I02	Submission with current validation failure	Submission is proposed while an executed validation check on the current revision has failed, with no later matched pass. A source change invalidates the earlier result rather than establishing success; outdated coverage belongs to F01.	G*	IS C1.3; TA §III-B3; Derived
I03	Work after verified completion	A trusted, explicitly defined completion condition is satisfied, but further non-required work is proposed or executed. Required cleanup is exempt; an unsupported success claim is not a completion oracle.	G*	SP §3.2.1; MA FM-1.5; Derived predicate
I04	Resource limit pressure or exhaustion	Measured token, context, output, step, cost, time, or memory usage reaches an actual limit or an explicitly declared reserve. Record the resource and measurement; action repetition is a possible contributor, not another resource failure.	G*	TR §3.2; AE App. A.2; Derived accounting

S Learned Behavioral Drifts

* These 23 patterns concern the correctness of beliefs, interpretations, repairs, tests, or objectives. A model can learn to recognize these patterns from examples. Observable process patterns can guide that judgment, and some cases require ground truth such as a reference patch or expected behavior.

Leave a judgment unresolved when evidence is unreliable, required context is unavailable, or support is too uncertain.

ID	Behavioral Drift Pattern	Possible observable proxy	Why process evidence is insufficient	Source
S01	Incorrect fault localization	Dispersed exploration or edits far from the initial investigation.	Repository distance does not identify the true faulty location.	IS §5.2 A; CA §2.2

S02	Misunderstood task requirements	Skipped obligations or unexpected feature scope.	The task's intended meaning must be established before judging understanding.	SP §3.2.1; AR §2.2
S03	Incorrect repair logic	Persistent failures or repeated patch revisions.	A logically wrong repair can follow a normal process and pass incomplete tests.	IS §5.2 B; CA §2.2
S04	Incomplete repair	Limited checks or missing related edits.	Completeness depends on required behavior. When the gap is a specific missed dependency, use S18.	IS §5.2 B; CA §2.2
S05	Overfitting to observed tests	Repeated targeted fixes or narrow validation.	Passing selected tests does not establish that the repair generalizes.	IS §5.2 B; TR §3.1
S06	Error masking or evasive source repair	Added suppression, broad exception handling, or fallback behavior.	Distinguishing legitimate handling from hiding a defect requires examining the code's behavior and the task requirements. Test-oracle weakening is S16.	IS §5.2 B
S07	Incorrect API or environment model	Invalid calls or repeated setup errors.	Observed errors do not establish the agent's mistaken belief about the interface or environment.	TR §3; AR §2.2
S08	False premise or causal misdiagnosis	A diagnosis is repeated despite apparently inconsistent evidence.	Assess the premise or causal explanation against the relevant evidence. Use S10 for a misread tool result and S12 for inconsistent beliefs or beliefs carried over from an earlier state.	FP §III-B; AE App. A.2
S09	Hallucinated action or result	A claim describes an execution or result absent from the recorded trace.	Distinguishing fabricated evidence from an inaccurate interpretation of a real result requires claim extraction and sufficiently complete logs.	TR §3.1; AR §2.2
S10	Misinterpreted tool feedback	The agent draws a conclusion inconsistent with an actual tool result.	Understanding arbitrary output and the agent's stated conclusion requires interpreting the output in context; fabricated results are S09.	TA §III-C4; AR §2.2

S11	Thought–action intent mismatch	A tool operation appears different from the stated next step.	Different wording or parameters do not establish that an action fails to implement the intention.	TA §III-C1; MA FM-2.6
S12	Contradictory or stale beliefs	Conflicting diagnoses, or repeated conclusions after state changes.	Logical contradiction, equivalent meaning, and relevance of new evidence require interpretation.	TA §III-C2; IS C3
S13	Failure to incorporate evidence	New results are followed by similar reasoning or actions.	The same action may still be appropriate; repetition does not prove the evidence was ignored.	TA §III-C4; WI §2.1.3
S14	Deviation from the task goal	Work shifts toward distant modules or secondary objectives.	Only the task specification establishes whether that work is irrelevant.	SP §3.2.1; WI §2.1.1
S15	Unjustified code changes	Large or dispersed changes, or repeated rework.	Assess whether changes are necessary for the intended task. Work that pursues a different objective belongs to S14.	WI §2.1.1; MS RQ2
S16	Incorrect or weakened test oracle	Assertions, fixtures, or test scope change after failure.	Test corrections can be legitimate; deciding whether an oracle was weakened requires expected behavior.	IS C2.3; MA FM-3.3
S17	Submission before meeting task requirements	Submission follows limited checking or unresolved evidence.	Process milestones do not establish that all task requirements have been fulfilled.	SP §3.2.1; MA FM-3.1
S18	Missed software dependencies	Uninspected related files or later integration failures.	Identifying required related changes requires understanding how the code behaves or consulting an external dependency model.	IS §5.2 B
S19	Inappropriate tool selection	A well-formed permitted call appears unhelpful for the subtask.	Interface validity and permission do not establish that a tool is suitable for the objective.	TR §3.1; SP §3.2.1
S20	Unresolved ambiguity in user intent	An ambiguous requirement is followed by an arbitrary choice without clarification.	Whether clarification is necessary depends on the task, available context, and consequences of the choice.	AR §2.2; MA FM-2.2

S21	Violation of a natural-language instruction or plan	Behavior omits or contradicts a stated obligation despite an otherwise appropriate task goal.	Judging adherence to a free-text requirement requires interpreting the instruction and behavior. A wrong understanding of the task is S02; pursuing another goal is S14.	WI §2.1.1; AR §2.2; MA FM-1.1
S22	Unsupported task capability	Execution repeatedly seeks an operation that the available interfaces do not provide.	Establishing that no available tool or combination can meet the request requires a task and capability model. Choosing a poor tool when a suitable one exists is S19.	AR §2.2
S23	Relevant evidence not retrieved	The agent makes a decision without consulting available information needed for the task.	Assess whether the information was relevant and necessary for the decision, using external evidence when needed. Not obtaining evidence differs from misinterpreting or ignoring evidence already observed, S10/S13.	TR §3.1; AE App. A.2

Boundaries between related signals

Related concepts	Distinction and coding rule
Action revisit and oscillation	A01 includes repeated occurrences of a single action identity. Use A02 for a recurring sequence of distinct action identities and retain its constituent revisits as attributes.
Action recurrence and repeated views	Action identity and structural region are different units. A changed view command can revisit the same region. Code a repeated inspection under B01 and retain action-identity recurrence as an attribute when both refer to that inspection.
Outdated and failed validation	F01 requires earlier validation followed by uncovered source changes. I02 requires an observed failure applicable to the current revision. Match each result to its revision.
Execution failure and test failure	D02 concerns setup or external execution errors. F04–F05 require executed tests and comparable results. A check that did not run does not establish a failed assertion or successful validation.

Test edits, selection changes, and oracle weakening	F06 records edits to test evidence. F07 records omission through a changed invocation selection. S16 judges whether the resulting oracle is wrong or weaker; an observable edit or omission does not establish that judgment.
Explicit contracts and task relevance	H01–H03 require declared, machine-checkable restrictions or invariants. S14 and S15 judge whether otherwise permitted work serves the actual task and is necessary.

Literature coverage

Literature concepts	Taxonomy location
Graphectory inefficiency patterns [GR, Table 3]	Navigation: B01–B04. Editing: E01–E05. Published names are retained; temporal and state refinements are identified in the source column.
Workflow and thought–action–result studies [PA, TA]	Workflow prerequisites and phase structure: C and F01–F03. Intent alignment, feedback interpretation, and evidence integration: S10–S13.
Failure taxonomies and recovery studies [WI, TR, IS, SP, MA, AR, AE, FP]	Tool failures and constraints: D and H; repair and validation evidence: E and F; completion and resource use: I; learned behavioral drifts: S.
Memory, evidence, and task support [MA, TR, AR, AE]	Recorded context loss: H06. Stale beliefs, ignored evidence, unretrieved evidence, ambiguous intent, unsupported capability, and instruction adherence: S12–S13, S20–S23. A visible context change is not proof of cognitive forgetting.
Provenance, runtime enforcement, and repository evidence [PG, AS, MS]	Explicit restrictions, reference origins, and state invariants: H; completion contracts: I; patch-shape candidates: E06–E07. PR-level findings motivate candidates rather than establish online failure judgments.
Limits of process evidence [CA, SV]	The learned behavioral drifts in S can occur without a distinctive process pattern. The catalogue summarizes patterns motivated by prior trajectory studies.

Sources

Section locators in the family tables refer to the works below. “Derived” predicates are proposed operationalizations; a citation supplies the motivating phenomenon, not a claim of implementation or intervention efficacy.

[GR] Process-Centric Analysis of Agentic Software Systems. OOPSLA 2026. §2.3, §3.3–3.4, Table 3.

https://doi.org/10.1145/3798271

[PA] From Plan to Action: How Well Do Agents Follow the Plan? ASE 2026. §§2,5.

https://arxiv.org/abs/2604.12147

[TA] Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. ASE 2025. §§III-B3, III-C.

https://arxiv.org/abs/2506.18824

[IS] An Empirical Study on Failures in Automated Issue Solving. §5.2 and Figures 6–8; A = localization, B = repair, C = iterative verification.

https://arxiv.org/abs/2509.13941

[SP] When Agents Go Astray: Course-Correcting SWE Agents with PRMs. 2025. §3.2.1.

https://arxiv.org/abs/2509.02360

[WI] Wink: Recovering from Misbehaviors in Coding Agents. AIware 2026. §2.1.

https://doi.org/10.1145/3805760.3814911

[TR] TRAIL: Trace Reasoning and Agentic Issue Localization. 2025. §3 and Figure 1.

https://arxiv.org/abs/2505.08638

[MA] Why Do Multi-Agent LLM Systems Fail? MAST. 2025. §4 and Appendix A.

https://arxiv.org/abs/2503.13657

[FP] Failure as a Process: An Anatomy of CLI Coding Agent Trajectories. 2026. §III-B.

https://arxiv.org/abs/2607.09510

[AR] AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. 2026. §§2.2,3.2.

https://arxiv.org/abs/2602.02475

[CA] Cadence: Strategic Guidance for Coding Agents. 2026. §2.2.

https://arxiv.org/abs/2610.12269

[AS] AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. ICSE 2026. §3.

https://doi.org/10.1145/3744916.3764546

[PG] Safeguarding LLM Agents from Misalignment through Provenance Analysis. 2026. §§4.2–4.3.

https://arxiv.org/abs/2607.01236

[MS] Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub. MSR 2026. RQ1–RQ2.

https://doi.org/10.1145/3793302.3793579

[AE] Where LLM Agents Fail and How They Can Learn From Failures. AgentErrorBench / AgentDebug. 2025. Appendix A.2.

https://arxiv.org/abs/2509.25370

[SV] A Survey for LLM Agent Trajectory Analysis: From Failure Attribution to Enhancement. IEEE TSE 2026. Background synthesis.

https://doi.org/10.1109/TSE.2026.3717765